

Overview

Jimi Open API empowers distributors and application vendors to provide tracking services to their customers by using their own GUI, app, and web clients, so they can localize their services and improve user experience.

Your App or web client should connect to your application server, rather than connect to JIMI API Server directly , use your application server connect to JIMI Server instead.

Usage

1. Contact us to apply your appKey and appSecret, you need to provide your account.
2. According to this document to implement your application server to obtain access_token.
3. Calling other interfaces with access_token to fulfil your business logic.

Conventions

Encoding, Format & Timezone

UTF-8 and JSON is used by default.

Context-Type:default application/json charset=utf-8

The API use UTC (GMT +0) time in default: format yyyy-MM-dd HH:mm:ss

Common response fields

Key	Type	Required	Description
code	int	Yes	result code
message	string	No	description

Result code description

code	Value	Description
-1	-1	The system is busy
0	0	success

code	Value	Description
1xxx	1001	Parameter error (lack of required parameters or format error). See interface description for details
	1002	Illegal user/illegal device (not their own or subordinate account or device)
	1003	Repeat operation
	1004	Illegal access, token exception! (Token failure or nonexistent)
	1005	Illegal access, ip access exceeds limit!
	1006	Illegal access, too frequently request!
	1007	Illegal access, request method error!
	1008	Illegal access, abnormal incoming!
	12001	Platform account creation failed
	12002	Device transfer failed
	12003	Geo-fence creation failed
	12004	Geo-fence deletion failed
	12005	Fail to send command

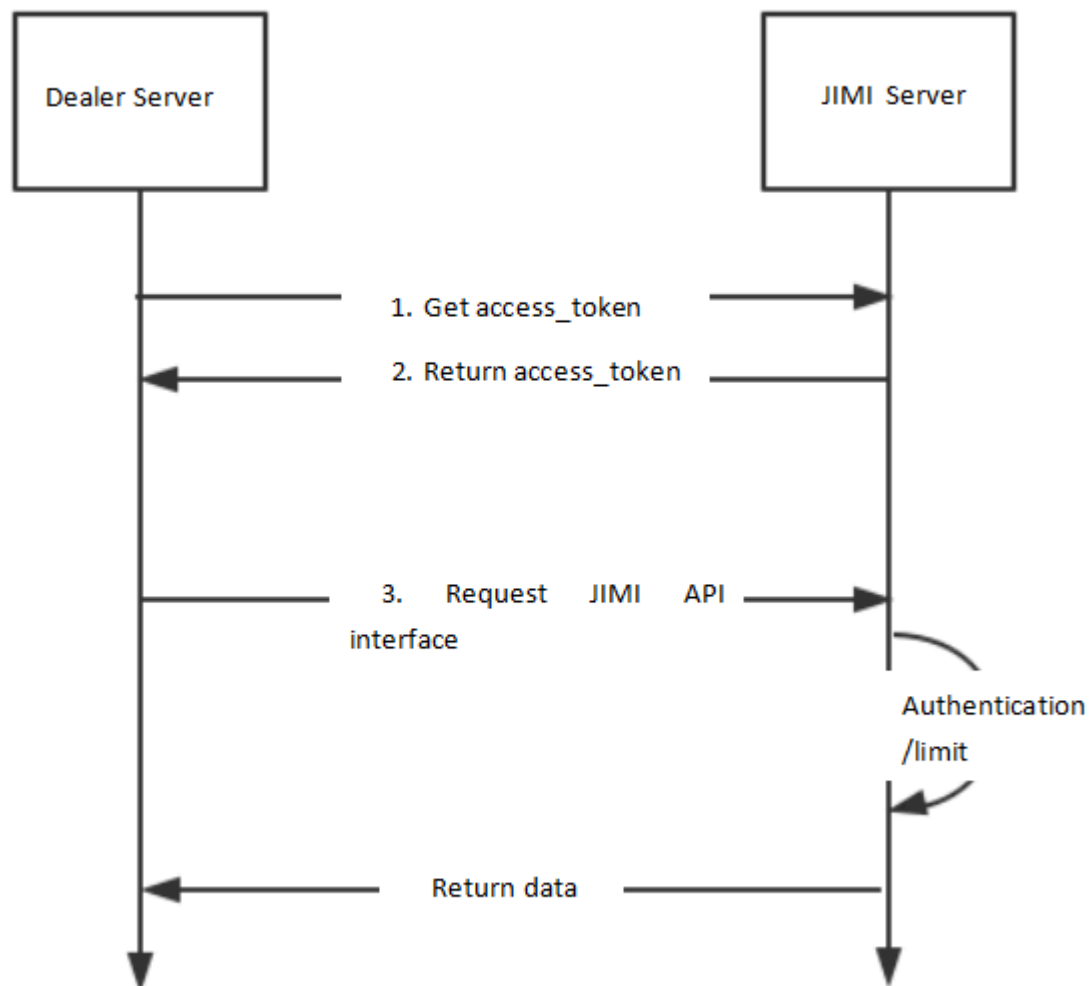
Error code :

code	Value	Description
Create account	213	Account already exist
	214	Account does not exist
	215	User type error. Possible reasons: 1.User type is empty; 2. User type does not exist; 3. Current login user do not have permission to create new type of user.
Device transfer	217	Target user does not exist
	218	User can only transfer/sale devices to sub-account
	219	IMEI list is illegal
	220	Number of IMEI exceeds limitation.

code	Value	Description
Geo-fence	41001	Exceed max number of Geo-fences supported
	41002	Fence name is already exists
	41003	The device is not online
	41004	Geo-fence operation failed
Command	225	Time out
	226	Parameter error
	227	The command is not executed correctly
	228	The device is not online
	229	Network error, connection error, etc.
	238	Device interrupted
	240	Data format error
	243	Not supported by device
	252	The device is busy

Work Flow

API Work Flow



API Features

Interface type	Method	Description
Access token	jimi.oauth.token.get	Get access_token
	jimi.oauth.token.refresh	Refresh access_token

Interface type	Method	Description
User API	jimi.user.child.list	List all sub-account
	jimi.user.device.list	List all devices of current account
	jimi.user.child.create	Create sub-account
	jimi.user.child.del	Remove sub-account
	jimi.user.child.move	Move sub-account to another account
	jimi.track.device.detail	Get device detail information for specific IMEI
	jimi.user.device.location.list	Get the latest location for all devices
Device API	jimi.device.location.get	Get the latest location data for specific IMEI
	jimi.device.location.URL.share	Get URL for showing location on the Map
	jimi.user.device.expiration.update	Change user expiration date for devices
	jimi.device.track.list	Obtain track data according to IMEI
	jimi.device.track.mileage	Get mileage report for devices
	jimi.open.device.update	Modify the vehicle information for device
	Jimi.device.media.URL	Get URL of photo or video capture by camera
Command API	jimi.device.live.page.url	Get device live streaming page URL
	jimi.open.instruction.list	Gets commands supported by the device
	jimi.open.instruction.send	jimi.open.instruction.send
Geo-fence API	jimi.open.instruction.result	Get the command result
	jimi.open.device.fence.delete	Remove the Geo fence for the device
LBS-API	jimi.open.device.fence.create	Create an Geo fence for the device
	jimi.lbs.address.get	Wi-Fi base station location analysis
Message	jimi.push.device.alarm	Push alarm notification
	jimi.device.alarm.list	Get alarm list for device
Scooter API	jimi.scooter.instruction.send	Send command to scooter device
	jimi.scooter.device.detail	Get scooter status and other detail information

API Description

Request URL

The unique request URL is: <http://open.10000track.com/route/rest> (<http://open.10000track.com/route/rest>)
Different interfaces are identified by request parameter method.

Security

Client should get access_token first before calling the interface, which is generated by JIMI Server by calling jimi.oauth.token.get interface(method=jimi.oauth.token.get) with provided appKey and appSecret.

appKey and access_token are required to sign request parameters when calling API. JIMI server will validate the the request parameters by checking the sign value.

Common parameterss

The parameters of each request should include Common parameterss and interface private parameters.
For example, if you call the “jimi.oauth.token.get” interface, you need to provide :

7(Common parameterss) + 3(private parameters) = 10 (parameters, key/value)

Name	Type	Required	Description	Remark	Default
method	string	Yes	API interface name		
timestamp	string	Yes	Timestamp, format:yyyy-MM-dd HH:mm:ss. Plus or minus10 minutes is allowed. e.g:2012-03-25 20:00:00	GMT(UTC) time.	
app_key	string	Yes	APP_KEY	from JIMI	
sign	string	Yes	A signature base on parameters, appKey, appSecret.		
sign_method	string	Yes	Optional,signature method. Available value: md5, support methods are: md5		

Name	Type	Required	Description	Remark	Default
v	string	Yes	Optional, specify the API version. System default 1.0, support version: 1.0	>md5	>md5
			System default 1.0, support version: 0.9, 1.0	1.0: check signature	
format	string	Yes	Optional, specify response format.		json

Signature

To protect API calling from hacked, any API calling needs to be with a signature. JIMI server will check signature based on request parameters. Illegal signature request will be rejected. Signature algorithms supported is: md5 (sign_method is a Common parameters mentioned above). Following is the algorithm of signature:

1. Sort all request parameters with parameter key in alphabetical order (including Common parameters and method specific parameters, but NOT include sign and byte type parameter.

For example:

```
foo=1, bar=2, foo_bar=3, foobar=4
Result: bar=2, foo=1, foo_bar=3, foobar=4
```

2. Remove all equal sign and comma:

```
bar2foo1foo_bar3foobar4
```

3. Then concatenate appSecret to the before and end of the result string and get the md5 value. E.g :

```
md5(<appSecret>+bar2foo1foo_bar3foobar4+<appSecret>), the real string should like:
md5(h9lri085eachcz4sn7gwnkh6j0jt0yz4bar2foo1foo_bar3foobar4h9lri085eachcz4sn7gwnkh6j0jt0yz4
)
```

Note that the string should be in UTF-8 encoding.

Note that the sign should be upper case string.

If the parameter value is a byte stream, it should be converted to hexadecimal. For example :

```
hex("helloworld".getBytes("utf-8")) = "68656C6C6F776F726C64"
```

MD5 is the 128-bit summary algorithm and is in hexadecimal. a hexadecimal character can represent four bits, so the signature string length is 32 hexadecimal characters.

JAVA example:

```

public static String signTopRequest(Map <String, String> params, >String seccode, String> s
    // 1: sort parameter key
    String[] keys = params.keySet().toArray(new String[0]);
    Arrays.sort(keys);

    // 2:: Put all parameter names and parameter values together
    StringBuilder query = new StringBuilder();
    if (Constants.SIGN_METHOD_MD5.equals(signMethod)) {
        query.append(seccode);
    }
    for (String key : keys) {
        String value = params.get(key);
        if (StringUtils.isNotEmpty(key, value)) {
            query.append(key).append(value);
        }
    }

    // 3: use MD5/HMAC to encrypt
    byte[] bytes;
    if (Constants.SIGN_METHOD_HMAC.equals(signMethod)) {
        bytes = encryptHMAC(query.toString(), seccode);
    } else {
        query.append(seccode);
        bytes = encryptMD5(query.toString());
    }

    // 4: convert binary to uppercase hexadecimal
    return byte2hex(bytes);
}

public static byte[] encryptHMAC(String data, String seccode) throws IOException {
    byte[] bytes = null;
    try {
        seccodeKey seccodeKey = new seccodeKeySpec(seccode.getBytes(Constants.CHARSET_UTF8)
        Mac mac = Mac.getInstance(seccodeKey.getAlgorithm());
        mac.init(seccodeKey);
        bytes = mac.doFinal(data.getBytes(Constants.CHARSET_UTF8));
    } catch (GeneralSecurityException gse) {
        throw new IOException(gse.toString());
    }
    return bytes;
}

public static byte[] encryptMD5(String data) throws IOException {
    return encryptMD5(data.getBytes(Constants.CHARSET_UTF8));
}

public static String byte2hex(byte[] bytes) {
    StringBuilder sign = new StringBuilder();
    for (int i = 0; i < bytes.length; i++) {
        String hex = Integer.toHexString(bytes[i] & 0xFF);
        if (hex.length() == 1) {
            sign.append("0");
        }
    }
}

```



```
    }  
    sign.append(hex.toUpperCase());  
}  
return sign.toString();  
}
```

Get access token

Description

Client(distributor's server) should stored the access_token locally, do NOT get access_token per request, instead, use the local access_token before it become invalid. JIMI server will not allow to access if the request frequency is too high from client.

The access token can be used for about 2 hours(depend on the value for expires_in parameter), do NOT try to get token for every request.

Request URL

See the unique request URL. Method = jimi.oauth.token.get

HTTP request method

POST

Request parameter

(1) Common parameters

(2) Private parameters

Name	Type	Required	Description	Remark	Default
user_id	string	Yes	User ID	User's account	
user_pwd_md5	string	Yes	user's password (md5)		Lowercase md5 value
expires_in	number	Yes	access token expired seconds	60-7200	

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result
accessToken	string	Access token, required by any other following interfaces
expiresIn	string	Access token expired seconds
account	string	User's account
appKey	string	AppKey from JIMI
refreshToken	string	Refresh token, which is used for updating accessToken
time	string	Token generated time

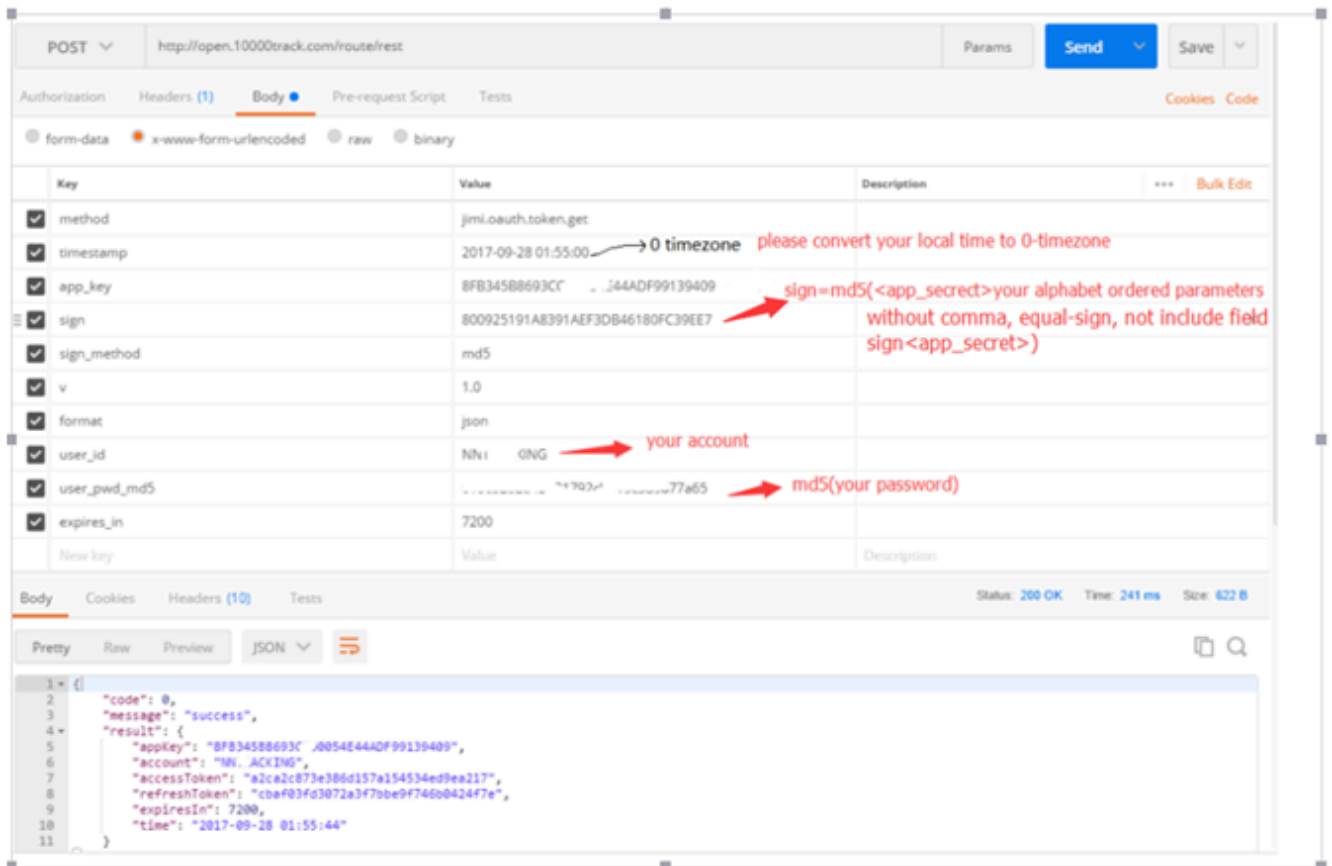
Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": {
    "appKey": "8FB345B8693CCD003CC2DAB61EC8791D",
    "account": "jimitest",
    "accessToken": "7da3330ec28e3996b6ef4a7e3390ba71",
    "expiresIn": 60
  },
  "refreshToken": "7da3330ec28e3996b6ef4a7e3390ba71",
  "time": "2017-06-15 10:00:00"
}
```

Return error example:

```
{"code":xxx,"message":"Incorrect user name or password"}
```

Postman demo:



user_id = your tracksolid account.

user_pwd_md5 = md5(your tracksolid account password).

Sign = md5(<app_secret>your alphabet ordered parameters keyvalue, without equal-sign, comma and not include sign field<app_secret>).

Note that sign is a 32-length upper case string.

For example:

```

<app_secret>app_key9FB345B8693CCD0054E44ADF99139409expires_in7200formatjsonmethodjimi.oauth.token.getsign_methodmd5timestamp2017-09-2801:55:00user_id<account>user_pwd_md5<password_md5>v1.0<app_secret>

```

Refresh access token

Description

This interface is used to update token manually when access token is about to be invalid.

Request URL

See the unique request URL. Method = jimi.oauth.token.refresh

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Description	Remark	Default
access_token	string	Yes	Access token		
refresh_token	string	Yes	Authorized refresh access token		
expires_in	number	Yes	Access token expired seconds	60-7200	

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result
accessToken	string	Access token, required for subsequently interface access.
expiresIn	string	Access token expired seconds.
account	string	Requested account
appKey	string	From JIMI
refreshToken	string	Refresh token for updating access token
time	string	Token generated time

Example for correct Response:

```
{
  "code": 0,
  "message": "success",
  "result": {
    "appKey": "8FB345B8693CCD003CC2DAB61EC8791D",
    "account": "jimitest",
    "accessToken": "7da3330ec28e3996b6ef4a7e3390ba71",
    "expiresIn": 60,
    "refreshToken": "7da3330ec28e3996b6ef4a7e3390ba71",
    "time": "2017-06-15 10:00:00"
  }
}
```

Return error example:

```
{"code":xxx,"message":"Illegal request, token is invalid"}
```

List all sub-account

Description

List all sub-account of a specified account.

Request URL

See the unique request URL. Method = jimi.user.child.list

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
target	string	Yes	-	The specified account for inquired.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result

Result:

Key	Type	Description
account	string	log in account
name	string	name
type	int	Account Type 3: App user 8: Distributor 9: ordinary users 10: ordinary distributor 11: sales
displayFlag	int	Available or not (1:Available,0:not available)
address	string	Location
birth	string	Birthday
companyName	string	Company name
email	string	Mailbox
phone	string	Contact number
language	string	Language (zh,en)
sex	int	Gender 0 male, 1 female
enabledFlag	int	Flag:1 Available, 0not available
remark	string	Remark

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "account": "123123",
      "name": "test",
      "type": 8,
      "displayFlag": 1,
      "address": null,
      "birth": "2017-04-22 00:00:00",
      "companyName": "",
      "email": "",
      "phone": "",
      "language": "zh",
      "sex": 0,
      "enabledFlag": 1,
      "remark": null
    }
  ]
}
```

Wrong return example:

```
{"code":xxx,"message":"The account does not exist"}
```

Create sub-account

Description

Create a sub-account of a specified account.

Request URL

See the unique request URL. Method = jimi.user.child.create

HTTP request method

POST

Request parameters

- (1) Common parameters
- (2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
super_account	string	No	-	The specified parent account. If not provided, use API account instead.
account_id	string	Yes	-	Account id, length of 3~30 characters. Support special characters like “_@.” as well as a-Z, 0-9.
nick_name	string	Yes	-	Nick name or customer name
account_type	int	Yes	-	1-Distributor 2-End User 3-Sales
password	string	Yes	-	Password with md5.
telephone	string	No	-	
Email	string	Yes	-	User could retrieve password if forgot it.
contact_person	string	No	-	
company_name	string	No	-	
permissions	string	Yes	-	Web Login App Login Send Command Set Working Mode Edit by Web Edit by App The 6 digits represent whether to the permission.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result

Correct return example :


```
{
  "code": 0,
  "message": "Vehicle information modification successful",
  "result": null
}
```

Wrong return example:

```
{"code":xxx,"message":"The account does not exist"}
```

Remove sub-account

Description

Remove a sub-account of a specified account.

Request URL

See the unique request URL. Method = jimi.user.child.del

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
super_account	string	No	-	The specified parent account. If not provided, use API account instead.
account_id	string	Yes	-	Account id, length of 3~30 characters. Support special characters like “_@.” as well as a-Z, 0-9.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result

Correct return example:

```
{
  "code": 0,
  "message": "Vehicle information modification successful",
  "result": null
}
```

Wrong return example:

```
{"code":xxx,"message":"no permissions"}
```

Move account

Description

Move account from one sub-account to another sub-account.

Request URL

See the unique request URL. Method = jimi.user.child.move

HTTP request method

POST

Request parameters

- (1) Common parameters
- (2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token for security access JIMI Server.
account	string	Yes	-	The account will be moved.
target_account	string	Yes	-	The target account to move in.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result

Correct return example :

```
{
  "code": 0,
  "message": "Vehicle information modification successful",
  "result": null
}
```

Wrong return example:

```
{"code":xxx,"message":"no permissions"}
```

List all devices of sub-account

Description

Query all devices of a specified account.

Request URL

See the unique request URL. Method = jimi.user.device.list

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
target	string	Yes	-	The specified account for inquired.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
imei	string	Device IMEI
deviceName	string	Device name
mcType	string	Device model
mcTypeUseScope	string	Automobile, electromobile, personal, pet, plane, others
sim	string	Sim card number
expiration	string	Platform expiration date
activationTime	string	Activation time
reMark	string	Remarks
vehicleName	string	Vehicle name
vehicleIcon	string	Vehicle icon
vehicleNumber	string	License plate number

Key	Type	Description
vehicleModels	string	Brand
carFrame	string	VIN
driverName	string	Driver name
driverPhone	string	Driver phone number
enabledFlag	int	Available or not (1:Available,0:not available)
engineNumber	string	Engine number

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "imei": "868120145233604",
      "deviceName": "868120145233604",
      "mcType": "GT300L",
      "mcTypeUseScope": "personal",
      "sim": "415451",
      "expiration": "2037-04-01 23:59:59",
      "activationTime": "2017-04-01 11:02:20",
      "reMark": "test",
      "vehicleName": null,
      "vehicleIcon": "bus",
      "vehicleNumber": "粤B3604",
      "vehicleModels": null,
      "carFrame": "2235",
      "driverName": "driver",
      "driverPhone": "13825036579",
      "enabledFlag": 1,
      "engineNumber": "8565674"
    }
  ]
}
```

Return error example:

```
{"code":xxx,"message":"Account queried doesn't exist"}
```

Get device detail information

Description

Get device detail information for specific IMEI.

Request URL

See the unique request URL. Method = jimi.track.device.detail

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
IMEI	string	Yes	-	The specified IMEI for inquired.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
imei	string	Device IMEI
deviceName	string	Device name
mcType	string	Device model
mcTypeUseScope	string	Automobile, electromobile, personal, pet, plane, others

Key	Type	Description
sim	string	Sim card number
expiration	string	Platform expiration date
user_expiration	string	User expiration date, format as: account1,2019-01-01 account2,2019-02-02
activationTime	string	Activation time
reMark	string	Remarks
vehicleName	string	vehicleName
vehicleIcon	string	Vehicle icon
vehicleNumber	string	License plate number
vehicleModels	string	Vehicle Model
carFrame	string	VIN
driverName	string	Driver name
driverPhone	string	Driver phone number
enabledFlag	int	Available or not (1:Available,0:not available)
engineNumber	string	Engine number
iccid	string	
importTime	string	
imsi	string	
licensePlatNo	string	
VIN	string	
vehicleBrand	string	
fuel_100km	string	Fuel consumption for per 100km.
status	string	0-disable 1-enable

Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": {
    "imei": "868120145233604",
    "deviceName": "868120145233604",
    "mcType": "GT300L",
    "mcTypeUseScope": "personal",
    "sim": "415451",
    "expiration": "2037-04-01 23:59:59",
    "activationTime": "2017-04-01 11:02:20",
    "reMark": "test",
    "vehicleName": null,
    "vehicleIcon": "bus",
    "vehicleNumber": "粤B3604",
    "vehicleModels": null,
    "carFrame": "2235",
    "driverName": "driver",
    "driverPhone": "13825036579",
    "enabledFlag": 1,
    "engineNumber": "8565674",
    "iccid": "xxxxxxx",
    "imsi": "xxxx",
    "importTime": "2017-04-01 11:02:20",
    "licensePlatNo": "8565674",
    "VIN": "xxxxxxx",
    "vehicleBrand": "xxxx",
    "fuel_100km": "9",
    "status": "8565674"
  }
}
```

Wrong return example:

```
{"code":xxx,"message":"Account queried doesn't exist"}
```

Get location of devices by account

Description

Get the latest location for all devices under an account.

Request URL

See the unique request URL. Method = jimi.user.device.location.list

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token for security access JIMI Server.
target	string	Yes	-	Specify the account for inquired.
map_type	string	No	-	map_type=GOOGLE, calibrated by google calibration. map_type=null, return origin latitude and longitude

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
imei	string	Device IMEI
deviceName	string	Device name
icon	string	Vehicle icon
status	string	Device status 0 - offline; 1 - online
Lat	double	Longitude (if the device is expired, the value is 0)
Lng	double	Latitude (if the device is expired, the value is 0)
expireFlag	string	Expired or not: 1- expired; 0 - not expired
activationFlag	string	Activate or not 1 - Activate; 0 - Not active
posType	string	GPS, LBS, WIFI,BEACON

Key	Type	Description
locDesc	string	Location information
gpsTime	string	GPS positioning time
hbTime	string	Heartbeat time
Speed	string	Speed(unit: km/h)
accStatus	string	ACC 0-OFF; 1-ON
electQuantity	string	Device battery (0-100), some models are not supported
powerValue	string	External voltage(0-100), some models are not supported

Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "imei": "868120145233604",
      "deviceName": "868120145233604",
      "icon": "bus",
      "status": "0",
      "posType": "GPS",
      "lat": 22.577282,
      "lng": 113.916604,
      "hbTime": "2017-04-26 09:14:50",
      "accStatus": "0",
      "speed": "0",
      "gpsTime": "2017-04-26 09:17:46",
      "activationFlag": "1",
      "expireFlag": "1",
      "electQuantity": "60",
      "locDesc": null
      "powerValue": null
    }
  ]
}
```

Wrong return example :

```
{"code":xxx,"message":"The account does not exist"}
```

Get the location of device(s)

Description

Get the latest location for one or multiple devices.

Request URL

See the unique request URL. Method = jimi.device.location.get

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		accesstoken: used to security access JIMI server.
imei	string	Yes	-	Device IMEI. Separate imei by comma; POST is recommended if too many devices (maximum 100 IMEI)
map_type	string	No	-	map_type=GOOGLE, calibrated by google calibration. map_type=null, return origin latitude and longitude

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data

Result list:

Key	Type	Description
IMEI	string	Device IMEI
deviceName	string	Device name
Icon	string	Vehicle icon
status	string	Device status 0, offline; 1, online
Lat	double	Longitude (if the device expires, the value is 0)
Lng	double	Latitude (if the device expires, the value is 0)
expireFlag	string	1- expired; 0 - not expired
activationFlag	string	1 - Activate; 0 - Not active
posType	string	GPS, LBS, WIFI, BEACON
locDesc	string	Location information
gpsTime	string	GPS positioning time
hbTime	string	Heartbeat time
speed	string	Speed (unit: km/h)
accStatus	string	ACC 0- off ;1- on
electQuantity	string	battery (0-100) , Some device models are not supported
powerValue	string	External voltage (0-100), Some device models are not supported
distance	string	Distance from device.

Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "imei": "868120145233604",
      "deviceName": "868120145233604",
      "icon": "bus",
      "status": "0",
      "posType": "GPS",
      "lat": 22.577282,
      "lng": 113.916604,
      "hbTime": "2017-04-26 09:14:50",
      "accStatus": "0",
      "speed": "0",
      "gpsTime": "2017-04-26 09:17:46",
      "activationFlag": "1",
      "expireFlag": "1",
      "electQuantity": "60",
      "locDesc": null
      "powerValue": null
    }
  ]
}
```

Wrong return example:

```
{"code":xxx,"message":"Illegal device"}
```

Get sharing location URL

Description

Get the URL of device location showing on the Map.

Request URL

See the unique request URL. Method = jimi.device.location.URL.share

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token: used to security access JIMI server.
imei	string	Yes	-	Device Imei

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data contains URL information.

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result":
    {
      "URL": "data.16180track.com/api/share?ver=2&method=trackDevice_abr&deviceinfo=7ae7c62"
    }
}
```

Wrong return example:

```
{"code":xxx,"message":"Illegal device"}
```

Update user expiration date of devices.

Description

Request URL

See the unique request URL. Method = jimi.user.device.expiration.update

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes		Access token: used to security access JIMI server.
imei_list	string	Yes	-	One or multiple devices IMEI.
new_expiration	string	Yes	-	New user expiration date for devices.

Response

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data contains URL information.

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "imei": "868120145233604",
      "update_result": "0",
      "update_msg": "update success"
    },
    {
      "imei": "868120145233605",
      "update_result": "1",
      "update_msg": "update failed, reason:xxxxxx"
    }
  ]
}
```

Wrong return example:

```
{"code":xxx,"message":"Illegal device"}
```

Get the mileage data of devices

Description

Get device(s) mileage data of specified time range.

Request URL

See the unique request URL. Method = jimi.device.track.mileage

HTTP request method

POST

(1) Common parameters

(2) Private parameters

Request parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imeis	string	Yes	-	Device imeis, separate by comma. Format: yyyy-MM-dd HH:mm:ss
begin_time	number	Yes	-	Start time Format: yyyy-MM-dd HH:mm:ss
end_time	number	Yes	-	End time Format: yyyy-MM-dd HH:mm:ss end_time should be earlier than current time
start_row	number	No	-	Row number of result set.
page_size	number	No	-	Records in one page.

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
imei	string	IMEI of device
startTime	string	Start time
endTime	string	End time
startLat	string	Latitude of start position.
startLng	string	Longitude of start position.
endLat	string	Latitude of end position.
endLng	string	Longitude of end position.
runTimeSecond	int	Second of elapsed between start and end position.
distance	Double	Distance(meter) between start and end position.
avgSpeed	Double	Average speed

E.g:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "imei": "3505831983422342",
      "startTime": "2017-04-26 00:00:58",
      "endTime": "2017-04-26 00:03:58",
      "startLat": 22.577144898887813,
      "startLng": 113.91674845964586,
      "endLat": 22.677144898887813,
      "endLng": 113.92674845964586,

      "elapsed": 2130,
      "distance": 25000,
      "avgSpeed": 90
    }
  ]
}
```

Get the track data of device

Description

Get device track data of not more than 2 days, within 3 months.

Request URL

See the unique request URL. Method = jimi.device.track.list

Request parameters

- (1) Common parameters
- (2) Private parameters

HTTP request method

POST

Request parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imei	string	Yes	-	Device imei(only 1 each time)
begin_time	number	Yes	-	Start time Format: yyyy-MM-dd HH:mm:ss
end_time	number	Yes	-	End time Format: yyyy-MM-dd HH:mm:ss end_time should be earlier than current time
map_type	string	No	-	map_type=GOOGLE, calibrated by google calibration. map_type=null, return origin latitude and longitude

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
lng	double	Longitude
lat	double	Latitude
gpsTime	string	GPS positioning time. Format yyyy-MM-dd HH: mm: ss
direction	string	Direction, polar coordinates started from due north
gpsSpeed	string	GPS speed
posType	string	1-GPS, 2-LBS, 3-WIFI
satellite	string	GPS antenna signal strength

Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "lat": 22.577144898887813,
      "lng": 113.91674845964586,
      "gpsTime": "2017-04-26 00:00:58",
      "direction": 0,
      "gpsSpeed": -1,
      "posType": 3
    },
    {
      "lat": 22.57708,
      "lng": 113.916631,
      "gpsTime": "2017-04-26 00:01:30",
      "direction": 184,
      "gpsSpeed": 0,
      "posType": 1
    }
  ]
}
```

Return error example:

```
{"code":xxx,"message":"IMEI does not exist{353419031939627}"}
```

```
{"code":xxx,"message":"The device has expired{353419031939627}"}
```

Update vehicle information by IMEI

Description

Update vehicle information by IMEI

Request URL

See the unique request URL. Method = jimi.open.device.update

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imei	string	Yes	-	Device IMEI
device_name	string	No	-	Device name
vehicle_name	string	No	-	Vehicle name
vehicle_icon	string	No	-	Vehicle icon
Vehicle plate number	string	No	-	Vehicle plate number
vehicle_models	string	No	-	Vehicle brand
driver_name	string	No	-	Driver name
driver_phone	string	No	-	Driver phone
device_statuses	string	No	-	0 - disable/1-enable. Enable/Disable devices.

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data,could be multiple rows.

Correct return example :

```
{
  "code": 0,
  "message": "Vehicle information modification successful",
  "result": null
}
```

Wrong return example:

```
{"code":xxx,"message":"imei doesn't exists"}
```

Vehicle Icon:

Icon key	Description
automobile	Car
bus	Bus
per	People
mtc	Motorcycle
truck	
taxi	
plane	
schoolBus	
excavator	
ship	
tricycle	
policeMtc	Police Motorcycle
tractor	
policeCar	
cow	
other	

Get Device Photo or Video URL

Description

Get device photo or video URL which capture by camera.

Request URL

See the unique request URL. Method = jimi.device.media.URL

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imei	string	Yes	-	Device IMEI
camera	string	Yes	-	1-front camera 2- inward camera
media_type	string	Yes	-	1 -photo 2-video
start_time	date	No	-	Start time of photo or video created.
end_time	date	No	-	End time of photo or video created.
token	string	No	-	Token used to validate whether can access photo or video or not.
page_no	int	No	-	Zero indexed, 0 by default.
page_size	int	No	-	10 rows by default.

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
thumb_URL	string	URL of video or photo thumbnail.
file_URL	string	URL of video or photo.
mime_type	string	Mime type of video or photo
create_time	date	Create time of video or photo.
media_type	string	1-photo 2-video
camera	string	0-front camera 1-inward camera
file_size	int	File size of photo or video.

Correct return example :


```
{
  "code": 0,
  "message": "Operation successful",
  "result":
  [
    {
      "thumb_URL": "Q9GLAFFqfCrYf6YfQAcON4w4Ezs=/lhWse7ie3wtFrjuQZ22dLAK5CSaR",
      "file_URL": "357730090345670_3949477_2019_04_29_12_32_38_01",
      "mime_type": "video/mp4",
      "create_time": 2019-04-29 10:32:43,
      "media_type": 2,
      "camera": 1,
      "file_size": 9949087
    },
    {
      "thumb_URL": "Q9GLAFFqfCrYf6YfQAcON4w4Ezs=/FhCrZEIy3518sR_y10lqDg7w3Ju-",
      "file_URL": "351609080120911_3949476_2019_04_29_18_32_12",
      "mime_type": "video/3gpp",
      "create_time": 2019-04-29 10:32:42,
      "media_type": 2,
      "camera": 1,
      "file_size": 2411318
    }
  ]
}
```

Return error example:

```
{"code":xxx,"message":"imei doesn't exists"}
```

Get Device Live Steaming Page URL

Description

Get device live streaming page URL& last position information.

Request URL

See the unique request URL. Method = jimi.device.live.page.url

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imei	string	Yes	-	Device IMEI
V	string	No	-	If v=2, need to provide token parameter, Jimi server will use this token to validate URL.
token	string	No	-	Token used to validate whether can access live video or not.

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
lng	double	Longitude
lat	double	Latitude
gpsTime	string	GPS positioning time. Format yyyy-MM-dd HH: mm: ss
direction	string	Direction, polar coordinates started from due north
gpsSpeed	string	GPS speed
posType	string	1-GPS, 2-LBS, 3-WIFI
satellite	string	GPS antenna signal strength
VIN	string	VIN
plateNo	string	License Plate Number
UrlCamera	string	Live streaming page URL

Correct return example:

```
{
  "code": 0,
  "message": "Vehicle information modification successful",
  "result": {
    "lat": 22.577144898887813,
    "lng": 113.91674845964586,
    "gpsTime": "2017-04-26 00:00:58",
    "direction": 0,
    "gpsSpeed": -1,
    "posType": 3,
    "satellite": 11,
    "VIN": "V12345",
    "plateNo": "ABC-12345",
    "UrlCamera": "https://www.domain.com/device/video/35408343202342345",
  }
}
```

Return error example:

```
{"code":xxx,"message":"imei doesn't exists"}
```

Wi-Fi, Base Station locating analysis

Description

Allocate by total devices under the account (10 times/day/device. All sub-accounts included)

Request URL

See the unique request URL. Method = jimi.lbs.address.get

HTTP request method

POST

Request parameters

- (1) Common parameters
- (2) Private parameters

Parameter	Type	Required	Remark	Description
-----------	------	----------	--------	-------------

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Access token: used to identify legal client.
imei	number	Yes	-	Device IMEI
lbs	number	No	-	(wifi/LBS: at least one) LBS information group (mcc,mnc,lac,cell,rssi), max 7. Each group has five, which should not be null and sorted in order. MCC, China: 460 MNC LAC information, 2312 23222 CELL code: 23222 RSSI Semaphore-70
wifi	string	No	-	(wifi/LBS: at least one) mac1,rssi1 mac2,rssi2 Mac address, no colon in between. Rssi signal strength

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
lng	string	Longitude
lat	string	Latitude
accuracy	string	Accuracy, the greater the value the better
direction	string	Direction, polar coordinates started from due north

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": {
    "lat": 40.65615416521587,
    "lng": 109.89894039833524,
    "accuracy": 0
  }
}
```

Return error example:

```
{"code":xxx,"message":"illegal device"}
```

Create Geo-fence for IMEI

Description

Create Geo-fence for IMEI

Request URL

See the unique request URL. Method = jimi.open.device.fence.create

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei
fence_name	string	Yes	-	Geo-fence name
alarm_type	string	Yes	-	Alarm type (in / out / in, out)

Parameter	Type	Required	Remark	Description
report_mode	string	Yes	-	Alarm reporting mode, 0: GPRS,1: SMS+GPRS
alarm_switch	string	Yes	-	Fence alarm switch(ON/OFF)
lng	string	Yes	-	Longitude
lat	string	Yes	-	Latitude
radius	string	Yes	-	Fence radius(1 ~ 9999; unit: 100 meters)
zoom_level	string	Yes	-	Zoom level (3-19)
map_type	string	Yes	-	Map (GOOGLE)

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data. Fence serial number returned if succeed.

Correct return example :

```
{
  "code": 0,
  "message": "Successfully create geo-fence.",
  "result": "5"
}
Wrong return example:
{
  "code": 41003,
  "message": "Device is not online, geo-fence creation failed ",
  "result": null
}
```

Delete Geo-fence for device

Description

Delete Geo-fence for device.

Request URL

See the unique request URL. Method = jimi.open.device.fence.delete

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei
instruct_no	string	Yes	-	Geo-fence command serial number

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data

Correct return example :

```
{
  "code": 0,
  "message": "delete the geo-fence successfully",
  "result": null
}
```

Return error example:

```
{
  "code": 41003,
  "message": "The device is not online and geo-fence can't be deleted",
  "result": null
}
```

Get command list supported by device

Description

Get command list supported by device

Request URL

See the unique request URL. Method = jimi.open.instruction.list

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result
id	string	Command code
orderName	string	Command name
orderContent	string	Command template
orderExplain	string	Command explanation
orderMsg	string	Prompt
isOffLine	string	If support offline command 0-no; 1-yes

Correct return example :

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "id": 81,
      "orderName": "SOS setting",
      "orderContent": "SOS,A,{0},{1},{2}#",
      "orderExplain": "SOS is used for receive alerts and SOS alerts. SOS number should have",
      "orderMsg": "",
      "isOffLine": "1"
    }
    ...
  ]
}
```

Return error example :

```
{"code":xxx,"message":"Illegal device"}
```

Send command to device

Description

Send command to device.

Request URL

See the unique request URL. Method = jimi.open.instruction.send

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei
inst_param_json	string	Yes	-	Command message json character string

inst_param_json description:

Parameter	Type	Required	Remark	Description
inst_id	string	Yes	-	Command code
inst_template	string	Yes	-	Command template
params	array	Yes	-	Command params string Array
is_cover	boolean	NO	false	Whether cover the existed offline command or not true-cover false-not cover

Example:

```
{ "inst_id": "113", "inst_template": "RELAY,1#", "params": [], "is_cover": "true" }
{ "inst_id": "114", "inst_template": "RELAY,0#", "params": [], "is_cover": "true" }
```

User-defined command:

```
{ "inst_id": "96", "inst_template": "{0}", "params": [ "STATUS#" ], "is_cover": "true" }
```

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	Result

Correct return example :

```
{
  "code": 0,
  "message": "command is successfully sent.",
  "result": null
}
```

Return error example:

```
{
  "code": 12005,
  "message": "Fail to send command. Result code: 226",
  "result": null
}
```

Get results of command execution

Description

Get results of sending command.

Request URL

See the unique request URL. Method = jimi.open.instruction.result

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
codeId	string	Command code
code	string	Command sent
content	string	Content replied by device
isExecute	string	Command status 0: execution failed, 1: successful execution, 3: to be sent, 4: canceled
isOffLine	string	Time, format: yyyy-MM-dd HH:mm:ss
sender	string	Sender
receiveDevice	string	Received imei
isOffLine	string	0: online 1: offline
idsource	string	Command description

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "codeId": "99",
      "code": "status#",
      "content": "Parameter error",
      "isExecute": "4",
      "sendTime": "2017-06-19 11:46:00",
      "sender": "jimitest",
      "receiveDevice": "86812011111117",
      "isOffline": "1",
      "idsource": "User-defined command"
    }
  ]
}
```

Return error example:

```
{"code":xxx,"message":"Illegal device"}
```

Receive Notification

Description

Third-party platform should provide an URL address to receive the notification(please contact us and provide the URL manually), JIMI server will push the notification by sending a HTTP request using this URL.

Message service List

Message Type	Description
jimi.push.device.alarm	Alarm data

Request content

Key	Type	Description
msgType	string	Message type, corresponding to the message service list
data	string	The content of the message, corresponding to msgType

Message content (jimi.push.device.alarm)

Alarms pushed are as follows:

Key	Type	Description
Type	string	Device imei
deviceName	string	String
alarmType	string	Alarm type
alarmName	string	Alarm name
lat	string	Latitude
lng	string	Longitude
alarmTime	string	Alarm time, format (yyyy-MM-dd HH: mm: ss)

E.g:

```
Parameter1:  
Key = msgType  
Value = jimi.push.device.alarm  
  
Parameter2:  
Key = data  
Value = {  
    "imei": "868120145233604",  
    "deviceName": "868120145233604",  
    "alarmType": "2",  
    "alarmName": "Power off alarm",  
    "lat": 40.65615416521587,  
    "lng": 109.89894039833524,  
    "alarmTime": "2017-05-08 12:00:00"  
}
```

Get device alarm list

Description

Get device alarm list.

Time range of searching criteria should be within 1 month.

Return maximum 1000 rows.

Request URL

See the unique request URL. Method = jimi.device.alarm.list

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei
alertTypeId	string	No	-	If not specify alertTypeId, all alert type will return.
begin_time	date	No	-	If not provide begin_time & end_time, returns latest 50 alerts of last 1 month.
end_time	date	No	-	

Response:

Key	Type	Description
code	int	Return code: 0: return correctly Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data
DeviceName	Device Name	Command code
IMEI	string	IMEI
model	string	Device Model
account	string	Account
account	string	Account

Key	Type	Description
account	string	Alert type
alertTime	string	AlertTime
positioningTime	string	Time of alert positioning
lng	double	Longitude
lat	double	Latitude

Correct return example:

```
{
  "code": 0,
  "message": "success",
  "result": [
    {
      "deviceName": "ABC-34352",
      "IMEI": "343503422910345",
      "model": "GT06N",
      "account": "test1234",
      "alertTypeId": "1002",
      "alertType": "ACC On",
      "alertTime": "2019-03-14 14:02:03",
      "positioningTime": "2019-03-14 14:02:03",
      "lat": 22.577144898887813,
      "lng": 113.91674845964586
    }
  ]
}
```

Return error example:

```
{"code":xxx,"message":"Illegal device"}
```

Send command (for scooter device)

Description

Send command to scooter device.

Request URL

See the unique request URL. Method = jimi.scooter.instruction.send

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	Device Imei
cmd_code	string	Yes	-	Command send to device, for example: To enable scooter, cmd_code can be set as: ENABLE# To lock scooter, cmd_code set as:LOCK# Note, sign # is required.

Response:

Key	Type	Description
code	int	Return code: 100: Success 200: Invalid parameter 300: Device is offline 301: Device unregistered 400: Network unavailable 600: HTTP request timeout
message	string	Description of sending command result.

Success example:

```
{
  "code": 100,
  "message": "Success"
}
```

Success example:

```
{
  "code": 200,
  "message": "Invalid parameter"
}
```

Get detail information of scooter

Description

Get detail information and status of device.

Request URL

See the unique request URL. Method = jimi.scooter.device.detail

HTTP request method

POST

Request parameters

(1) Common parameters

(2) Private parameters

Parameter	Type	Required	Remark	Description
access_token	string	Yes	-	Accesstoken: used for identifying legal third party
imei	string	Yes	-	IMEI of scooter, multiple scooter Separated by commas, for example: 350583223422451,350583223422452

Response:

Key	Type	Description
code	int	Return code: 0: return success Other: failure. Refer to the error code description
message	string	If code is not 0, there will be a corresponding error message
result	string	The returned data(json)

Result list:

Key	Type	Description	
deviceImei	string	IMEI of IoT module	
realtimeStatus	json	DLCC	DLCC status 0 - OFF 1 - ON
		brake	Brake status 0 - OFF 1 - ON
		settingBrakeLight	Setting brake light status 0 - OFF 1 - ON
		settingFrontLight	Setting front light status 0 - OFF 1 - ON
		realFrontLight	realFrontLight front light status 0 - OFF 1 - ON
		realBrakeLight	Real brake light status 0 - OFF 1 - ON
		charge	Charge status 0 - OFF 1 - ON
		acceleratorStatus	0 - OFF 1 - ON
		motoStatus	0 - OFF 1 - ON
		workingMode	Driving mode. 0 - Disability mode 1 - Lock mode 2 - Enabling mode 3 - Charging mode 4 - Maintenance mode

Key	Type	Description	
realtimeData	json	batteryLevel	BAT level, for example, if BAT level = 50%, batteryLevel value is 50.
		current	Output current
		speed	Speed(Hall)
		voltage	BAT voltage
		batteryCycle	Battery cycle
		totalOdometer	Total mileage
		currentOdometer	Mileage of current trip(before disabled).
		batteryTemperature	°C
		remainingMileage	Remaining mileage
scooterProperties	json	ECUVersion	ECU version
		batteryCapacity	Battery capacity
		batteryVoltage	Scooter rated voltage
		firmwareVersion	Firmware version
		scooterVersion	scooterVersion
		speedLimit	Scooter maximum speed
		IoT	0 - OK 1-error

Key	Type	Description	
scooterError	json	accelerator	0 - OK 1-error
		battery	0 - OK 1-error
		brakeHandlebar	0 - OK 1-error
		controller	0 - OK 1-error
		dashboard	0 - OK 1-error
		voltage	0 - OK 1-error
		Motor	0 - OK 1-error
IoTModule	json	gpsStatus	0 - GPS not available 1 - GPS positioning
		course	Direction of positioning
		satellite	Satellite number
		gpsSpeed	Speed
		gpsTime	Positioning time
		positioningType	GPS LBS WIFI
		Latitude	Latitude
		gsmSignal	0 - No signal 1 - Extremely week 2 - Week 3 - Strong 4 - Extremely strong
		mCC	mCC
		mNC	mNC
		BluetoothMac	string
		iccid	string
		imsi	string
		batteryVoltage	Battery voltage

Correct return example:

```
{
  "code": 0,
  "msg": "Operation success.",
  "result": {
    "deviceImei": "860425040472944",
    "scooterError": {
      "IoT": 0,
      "accelerator": 0,
      "battery": 0,
      "brakeHandlebar": 0,
      "controller": 0,
      "dashboard": 0,
      "motor": 0,
      "voltage": 0
    },
    "scooterProperties": {
      "ECUVersion": "0.1",
      "batteryCapacity": 22834,
      "batteryVoltage": 36,
      "bluetoothVersion": "2.1",
      "firmwareVersion": "c.0",
      "scooterVersion": "3.0",
      "speedLimit": 50
    },
    "realtimeData": {
      "totalOdometer": 14592,
      "batteryCycle": 512,
      "batteryLevel": 56,
      "batteryTemperature": 23,
      "current": 614.39,
      "currentOdometer": 217.6,
      "remainingMileage": 69.2,
      "speed": 99.86,
      "voltage": 460.94
    },
    "realtimeStatus": {
      "DLCC": 0,
      "DLCCStatus": 0,
      "acceleratorStatus": 0,
      "brake": 0,
      "settingBrakeLight": 1,
      "charge": 0,
      "settingFrontLight": 0,
      "realFrontLight": 0,
      "motoStatus": 0,
      "realBrakeLight": 0,
      "workingMode": 0
    },
    "drivingSetting": {
      "DLCC": 0,
      "drivingMode": 2,
      "lightMode": 0,
      "quickStart": 1,
      "speedLimit": 30,

```

```
    "speedUnit": 0
  },
  "IoTModule": {
    "imsi": "0460045394000271",
    "iccid": "89860433231880390271",
    "BluetoothMac": "000000000000",
    "satellite": 11,
    "course": 0,
    "gpsMode": 0,
    "gpsSpeed": 0,
    "gpsTime": "2019-05-09 18:41:12",
    "lAC": 0,
    "latitude": 22.577137,
    "longitude": 113.916756,
    "mCC": 0,
    "mNC": 0,
    "positioningType": "GPS",
    "gsmSignal": 4,
    "gpsStatus": 0,
    "batteryVoltage": 4.15
  }
}
```

Success example:

```
{"code":xxx,"message":"Illegal device"}
```